SA

AGENT CONCEPTS + MCP BASICS

Tools give AI reach. Control flow decides whether it is an agent.

A plain-language classification of my FL-04 pipeline, one bounded MCP setup, and three tool calls that chat alone could not perform.

EVALUATOR MAP

Pass criteria, made inspectable.

Criterion	Evidence in this report	Status
Agent and workflow explained accurately	Control-flow test in §§1–2	Ready
FL-04 classified	Fixed five-stage pipeline = workflow	Ready
MCP and three primitives	Tools, resources, prompts in §3	Ready
Three tasks show tool use	Live screenshot gallery in §5	Add captures
Concrete agent upgrade	Evidence-verification loop in §4	Ready

01 · THE USEFUL DEFINITION

An agent chooses the route.

“Agent” should describe who controls the route, not how impressive the output sounds. A normal chat answers a prompt. A workflow may make several model calls or use tools, but its route is decided in advance. An agent receives a goal and decides what to do next: it can choose a tool, inspect the result, revise its plan, and repeat until it reaches a stopping condition or asks a human for help.

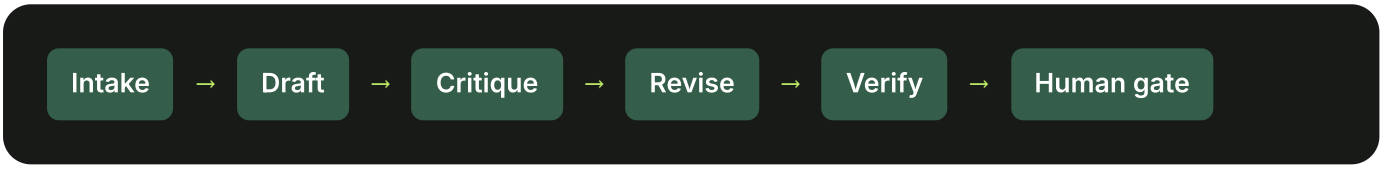
That makes autonomy a design choice, not a badge. Agents are useful when the correct steps cannot be known beforehand—for example, diagnosing a fault that may require searching different files depending on what each test reveals. They also cost more time and tokens, and their flexible paths are harder to predict and evaluate. For a stable, repeatable task, a workflow is often the stronger design.

---

02 · APPLIED TO FL-04

My pipeline is a workflow.

The clean test is control flow. In a **workflow**, the builder defines the path: step A hands to B, then C, with fixed gates. The model works inside that route. In an **agent**, the model directs the path while it runs. It chooses which actions are necessary from the evidence it observes.



My FL-04 Draft–Critique–Revise pipeline is a **workflow**. Its route is fixed before any input arrives: intake builds a fact ledger; drafting produces cited copy; critique checks four named risks; revision applies the issue log; verification returns a status and a human-check list. The same five stages run in the same order. Even the final human gate is predetermined. Claude generates content inside each stage, but it cannot decide to skip drafting, inspect a source repository, run a benchmark, or invent a new verification step. A multi-step prompt is not automatically an agent.

---

03 · THE CONNECTION LAYER

MCP standardizes access; it does not create autonomy.

The Model Context Protocol (MCP) is a standard way for an AI application to connect to capabilities outside the conversation. The host in my test is Codex. It connects through an MCP client to one or more servers, and each server exposes a clear interface. Instead of every AI product needing a custom integration for every file store or service, MCP gives them a shared protocol.

Tools

Executable operations the model can choose, such as reading a file, searching a directory, or creating a document.

Resources

Addressable context the application can supply, such as file contents, database records, or repository history.

Prompts

Reusable, user-selected instruction templates that package a known interaction.

The distinction matters because MCP is plumbing, not agency. Connecting a filesystem server gives the model hands, but it does not decide how much freedom the model should have. A fixed workflow can use MCP tools, and an agent can operate without MCP through another tool interface. MCP standardizes access; the surrounding system supplies goals, permissions, control flow, and stopping rules.

For my evidence setup, I connected Codex to the official Filesystem MCP server and limited it to this assignment folder. Three tests prove the connection: listing the allowed directory, reading an existing brief, and creating a new summary file. Plain chat could guess what files might contain, but it could not inspect the live directory or write a file there. The evidence must show Codex's filesystem MCP tool call as well as the returned result; a text answer alone is not proof.

---

04 · ONE CONCRETE UPGRADE

Add a bounded evidence-verification agent.

I would give it a goal—produce supportable portfolio copy—and a bounded tool set: read approved project files, search the repository, run named tests, and inspect benchmark outputs. After intake, the model would decide which claim needs checking, choose the relevant tool, observe the result, and either gather more evidence, weaken the claim, or ask me for a missing fact. It would stop only when every checkable claim had evidence or an explicit unresolved label.

That changes control flow instead of merely adding another prompt. A test failure could send the agent to the implementation and then back to the test; a missing source could trigger a human question; a verified claim could proceed directly to drafting. The current critique and final human approval should remain as guardrails. File access would be allow-listed, write actions would require approval, and publishing would stay outside the agent's permissions.

The upgrade would add flexibility where FL-04 is weakest: its fact ledger checks whether copy matches supplied facts, but not whether those facts are true or current. The agent could gather live evidence. I would only accept the extra complexity if evaluation showed better claim accuracy than the simpler workflow,

because autonomy is valuable when it resolves uncertainty—not when it merely makes a fixed pipeline harder to inspect.

---

05 · REQUIRED LIVE PROOF

Working MCP connection

> **Required:** upload the connection view and all three tasks. Each capture must show a Filesystem MCP tool call and its result—not only Codex's prose.

SETUP

Filesystem server connected

Server/tool list visible.

TASK 1

List live directory

Tool returns real filenames.

TASK 2

Read local brief

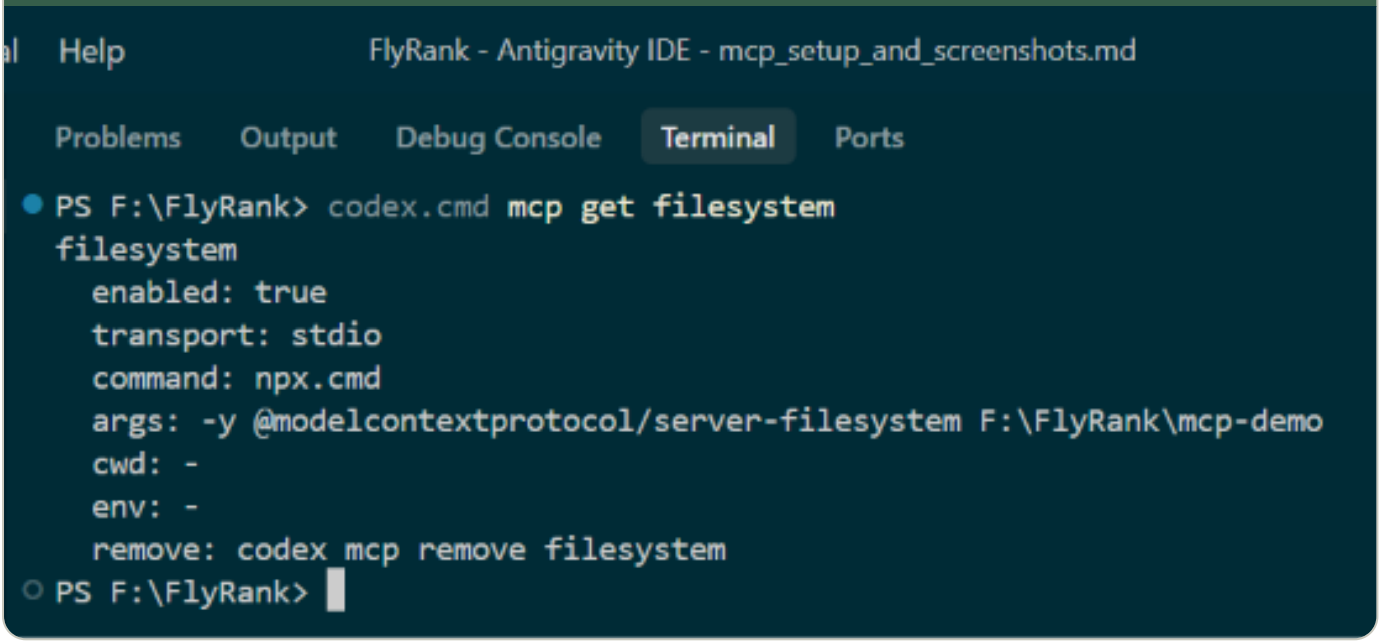
Tool returns file-only details.

TASK 3

Create + read back

Write and read tools visible.

MCP evidence 1 of 4 00-mcp-connected.png .png



MCP evidence 2 of 4 01-list-directory.png .png

MCP evidence 3 of 4 02-read-file.png .png

```
› Use only the connected Filesystem MCP server to read F:\FlyRank\mcp-demo\project-brief.txt. Return
  the project name, all fixed workflow stages, and the unresolved evidence gap. Show the Filesystem
  MCP tool call. Do not use the shell.

• I'll read that exact file through the Filesystem MCP and extract only the requested fields.

• Called filesystem.read_text_file({"path":"F:\\FlyRank\\mcp-demo\\project-brief.txt"})
  └ PROJECT: FL-04 Draft-Critique-Revise portfolio-copy pipeline
    FIXED STAGES: Intake, Draft, Critique, Revise, Verify & Format
    CONTROL FLOW: Every input follows the same five stages, then stops at a human approval gate.
    FACT BOUNDARY: Each checkable sentence must point to a supplied fact ID.
    UNRESOLVED EVIDENCE GAP: The workflow can check copy against supplied facts, but it cannot prove
        those facts are true or current.
    PUBLISHING RULE: Never auto-publish.

---

• Filesystem MCP tool call:

  read_text_file({
    "path": "F:\\FlyRank\\mcp-demo\\project-brief.txt"
  })

  - Project: FL-04 Draft-Critique-Revise portfolio-copy pipeline
  - Fixed workflow stages: Intake, Draft, Critique, Revise, Verify & Format
  - Unresolved evidence gap: The workflow can check copy against supplied facts, but it cannot prove
    those facts are true or current.

---
```

MCP evidence 4 of 4 03-create-and-read.png .png

SOURCES READ

Primary documentation

- Anthropic — Building Effective Agents
- MCP — Architecture overview
- MCP — Server primitives
- MCP — Connect to local servers

Sara Arif · General AI Fluency · Week 4 · Assignment 4 · FL-05